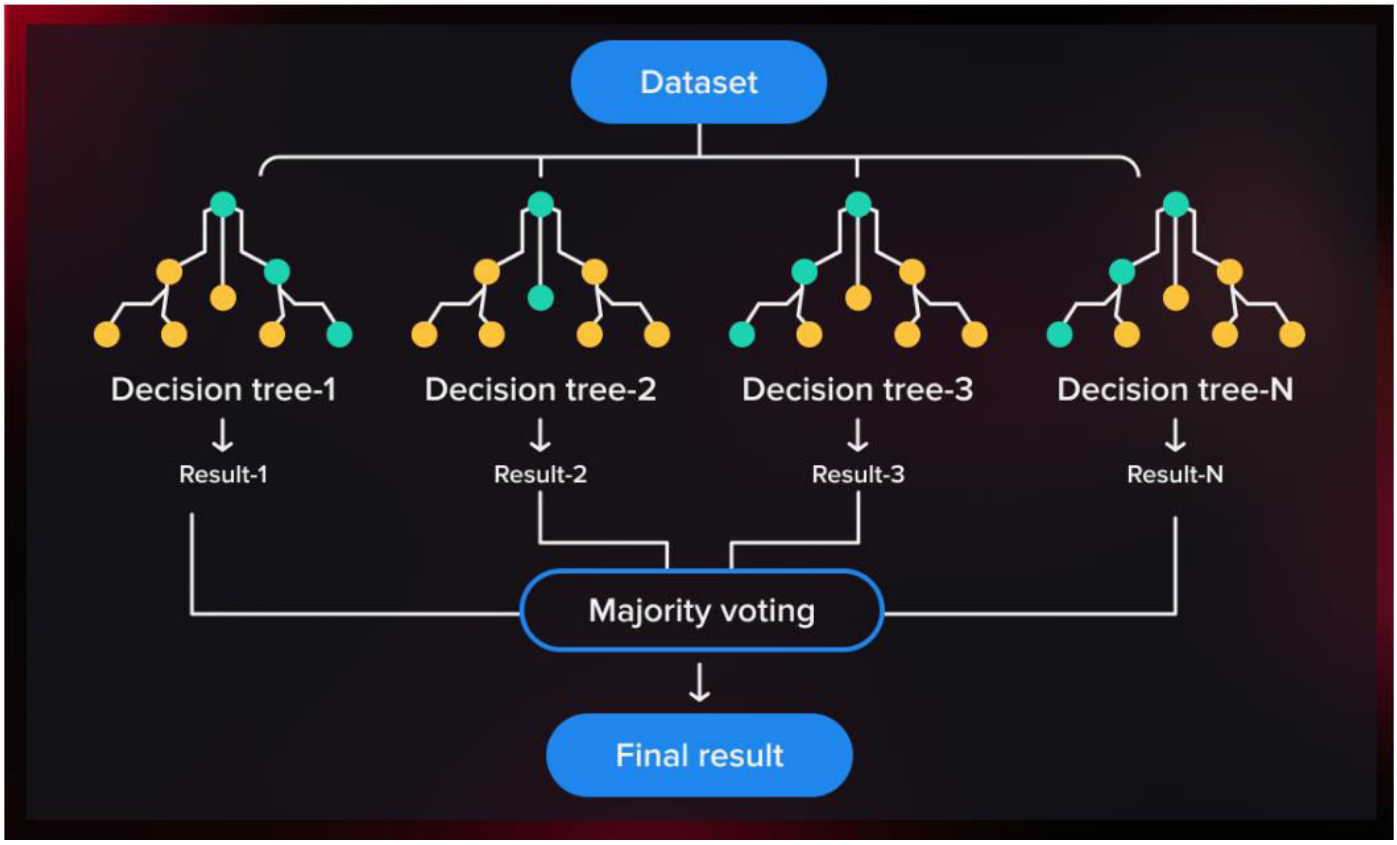
Random Forest & Bagging

Random Forest

What is Random Forest?

Random Forest is an ensemble learning method that builds multiple decision trees and combines their predictions to improve accuracy and reduce overfitting. It uses **Bagging (Bootstrap Aggregating)** to create diverse trees by training them on random subsets of data.



Key Characteristics of Random Forest

- **Ensemble of decision trees**: Uses multiple trees to make predictions.
- **Reduces overfitting**: By averaging multiple trees, variance is minimized.
- **Works for classification and regression**: Suitable for both tasks.
- **Handles missing values well**: Can still work with some missing data.
- **Feature importance**: Provides insights into which features are most influential.

How Random Forest Works?

1. **Random Subsets**: Each tree gets a **random subset** of training data using bootstrapping.
2. **Feature Randomness**: Each tree considers only a **random subset of features** when splitting nodes.

3. **Decision Trees Training**: Trees are trained independently.
4. **Voting/Averaging**:
   - **For Classification**: Majority voting decides the final class.
   - **For Regression**: The average of predictions from all trees is taken.

2. Feature Importance in Random Forest

Random Forest ranks feature importance based on how much each feature contributes to reducing impurity in decision trees.

Methods to Determine Feature Importance:

- **Mean Decrease in Impurity (MDI)**: Measures how much Gini impurity or entropy decreases by including a feature.
- **Mean Decrease in Accuracy (MDA)**: Measures the drop in accuracy when a feature is randomly shuffled.

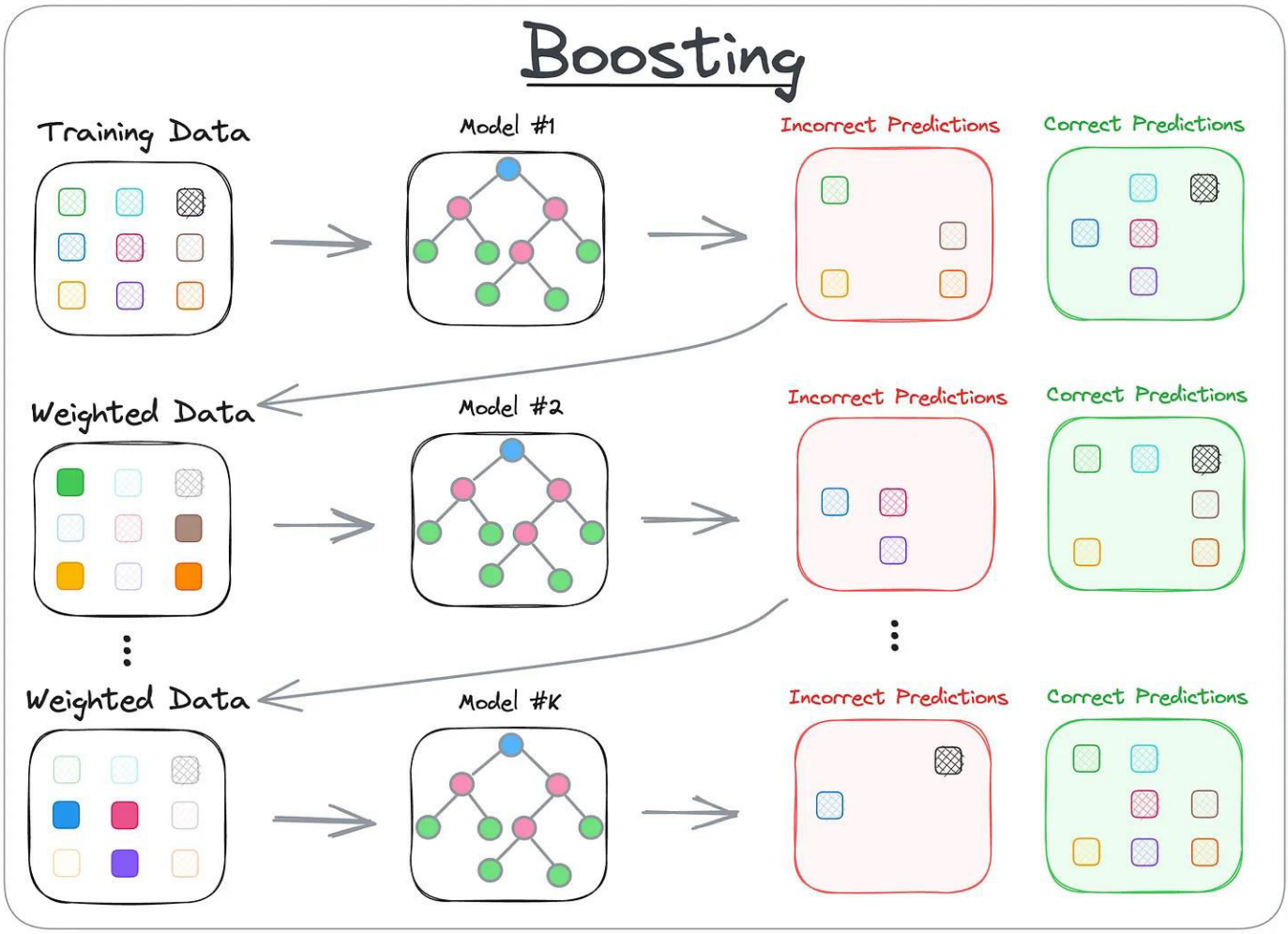
How to Get Feature Importance in Python?

```
from sklearn.ensemble import RandomForestClassifier
import pandas as pd
import numpy as np

# Sample dataset
X = np.random.rand(100, 5)
y = np.random.randint(2, size=100)

# Train Random Forest
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X, y)

# Get feature importance
feature_importance = pd.DataFrame({'Feature': range(X.shape[1]),
                                   'Importance': rf.feature_importances_})
print(feature_importance.sort_values(by='Importance', ascending=False))
```

AdaBoost (Adaptive Boosting)

What is AdaBoost?

AdaBoost is a boosting algorithm that improves weak learners (like shallow decision trees) by adjusting their weights based on previous errors.



How AdaBoost Works?

1. **Start with Equal Weights**: Assign equal weights to all training samples.
2. **Train a Weak Learner**: A small decision tree (stump) is trained.
3. **Update Weights**: Misclassified points get **higher weights**, so the next tree focuses on them.
4. **Repeat**: Multiple weak learners are combined using weighted voting.

Advantages of AdaBoost

- **Better generalization** than single decision trees.
- **Focuses on hard-to-classify examples**.
- **Works well with small datasets**.

Python Implementation of AdaBoost

```python
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier

# Weak learner (stump)
stump = DecisionTreeClassifier(max_depth=1)

# Train AdaBoost
ada = AdaBoostClassifier(base_estimator=stump, n_estimators=50, learning_rate=1.0)
ada.fit(X, y)

# Predictions
y_pred = ada.predict(X)
```

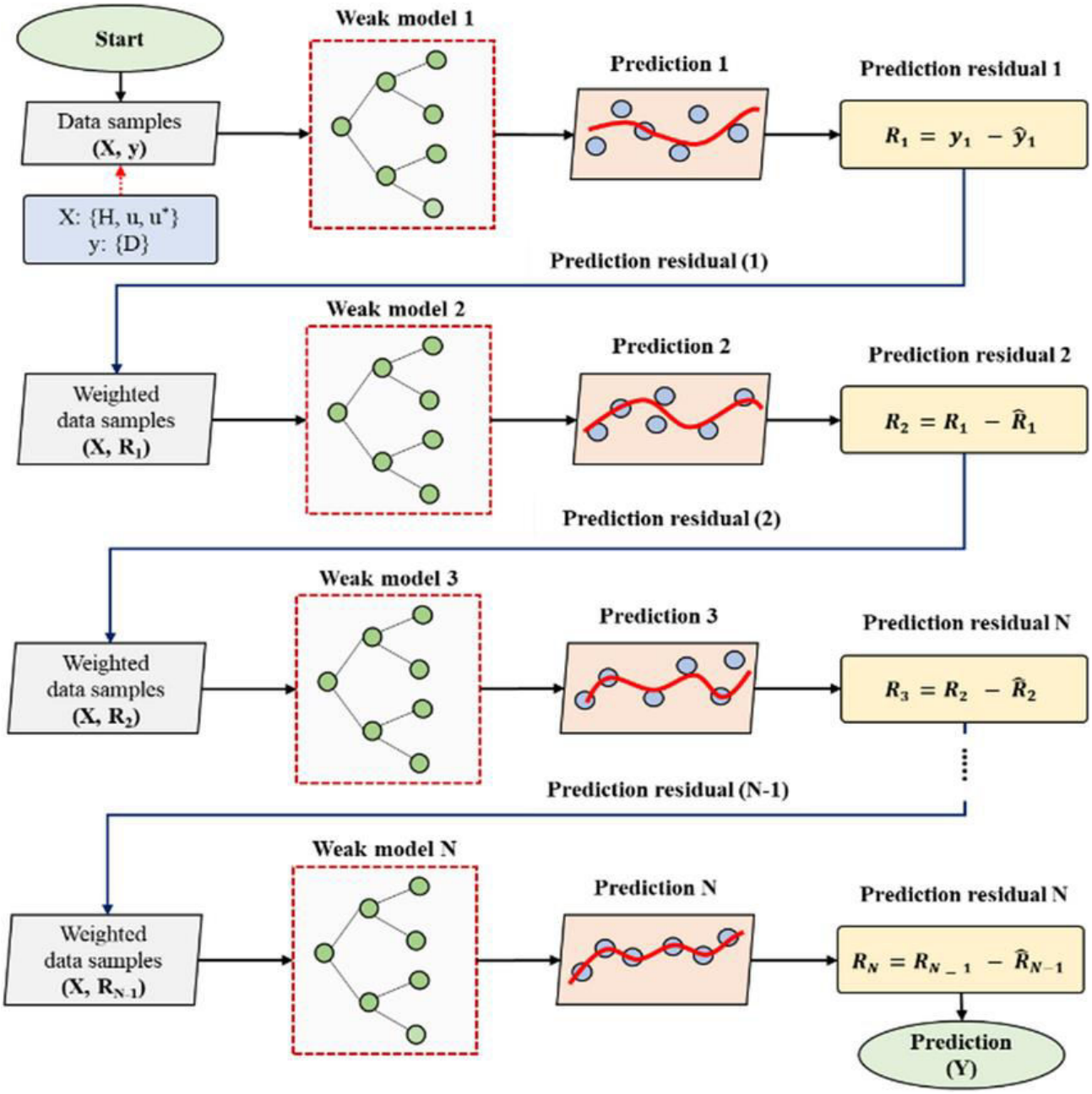
4. Gradient Boosting (GBM, XGBoost, LightGBM, CatBoost)

What is Gradient Boosting?

Gradient Boosting improves weak learners iteratively by correcting previous errors using gradient descent. Unlike AdaBoost, it optimizes residual errors.

How Gradient Boosting Works?

1. **Train Weak Model**: Start with a weak decision tree.
2. **Calculate Residuals**: Find the difference between predictions and actual values.
3. **Train Next Model on Residuals**: The next tree learns to fix previous errors.
4. **Combine Predictions**: Models are combined using weighted summation.



Popular Gradient Boosting Variants

- **XGBoost**: Optimized for speed and performance.
- **LightGBM**: Faster training using histogram-based approach.
- **CatBoost**: Handles categorical features efficiently.

Python Implementation of Gradient Boosting

```
from sklearn.ensemble import GradientBoostingClassifier

gbm = GradientBoostingClassifier(n_estimators=100, learning_rate=0.1, max_depth=3)
gbm.fit(X, y)

y_pred = gbm.predict(X)
```

5. Voting Classifier

What is a Voting Classifier?

A Voting Classifier combines predictions from multiple models (e.g., Decision Trees, SVM, Logistic Regression) to improve accuracy.

Types of Voting:

- **Hard Voting**: Takes the most common prediction.
- **Soft Voting**: Takes the average probability of all models.

Python Implementation of Voting Classifier

```
from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC

lr = LogisticRegression()
svm = SVC(probability=True)
rf = RandomForestClassifier()

voting_clf = VotingClassifier(estimators=[('lr', lr), ('svm', svm), ('rf', rf)], voting='soft')
voting_clf.fit(X, y)

y_pred = voting_clf.predict(X)
```

6. Bagging vs. Boosting

Feature	Bagging	Boosting
Concept	Parallel training of multiple models	Sequential training with error correction
Example	Random Forest	AdaBoost, Gradient Boosting
Overfitting	Reduces overfitting	Can overfit with high learning rate
Weak Learners	Independent decision trees	Adjusts weights to focus on misclassified examples

Feature	Bagging	Boosting
Bias-Variance	Reduces variance	Reduces bias

Interview Questions & Answers

Q1. What is the difference between Random Forest and Gradient Boosting?

A:

- **Random Forest** is a **bagging** method that builds independent decision trees and averages results.
- **Gradient Boosting** is a **boosting** method that builds trees sequentially, correcting previous errors.

Q2. How does AdaBoost differ from Gradient Boosting?

A:

- **AdaBoost** adjusts weights of misclassified samples.
- **Gradient Boosting** corrects errors using gradient descent.

Q3. What is the advantage of using Voting Classifier?

A: It improves model performance by combining different algorithms to leverage their strengths.

Q4. When should you use Bagging vs. Boosting?

A:

- Use **Bagging (Random Forest)** when reducing **variance** is needed.
- Use **Boosting (Gradient Boosting, AdaBoost)** when reducing **bias** is the goal.

Q5. What hyperparameters are important for Random Forest?

A:

- **n_estimators**: Number of trees.
- **max_depth**: Maximum depth of trees.
- **max_features**: Number of features per tree.
- **min_samples_split**: Minimum samples to split a node.

Practice Links & Blogs

📎 **Random Forest (Scikit-Learn Docs)**: https://scikit-learn.org/stable/modules/ensemble.html#forest

📎 **Gradient Boosting (XGBoost Guide)**: https://xgboost.readthedocs.io/en/latest/tutorials/model.html

📎 **AdaBoost (GeeksforGeeks)**: https://www.geeksforgeeks.org/adaboost/

📎 **Bagging vs Boosting (Machine Learning Mastery)**: https://machinelearningmastery.com/bagging-and-boosting-ensemble-methods-for-machine-learning/